Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Системы автоматизированного проектирования»

ОТЧЁТ

по лабораторной работе № 2
по дисциплине «Логика и основы алгоритмизации в ИЗ» на тему «Оценка времени выполнения программ»

Выполнили студенты гр. 25BBB4:
Поликарпов Я. Егоров Е.
Приняли:
Юрова О. В. Деев М. В.

Пенза 2026

Задание 1

Цель - определить порядок сложности программы умножения квадратных матриц, измерить время её выполнения и сопоставить эксперимент с теоретической оценкой.

В программе матрицы хранятся в динамически выделенных одномерных массивах. Это исключает переполнение стека при больших N. Для каждого элемента результата выполняется внутренний цикл суммирования из N произведений.

Сложность этапов: выделение памяти - $O(N^2)$, заполнение двух матриц - $O(N^2)$, умножение - $O(N^3)$, освобождение памяти - $O(1)$. Следовательно, итоговая асимптотическая сложность программы определяется умножением: $O(N^3)$. Потребление памяти составляет $O(N^2)$ для трёх матриц.

Измерение выполнено функцией clock(). В столбце «общее время» дополнительно учтены выделение памяти, заполнение случайными числами и освобождение памяти; «умножение» измеряет только тройной вложенный цикл. Размеры 100, 200, 400, 1000, 2000 и 4000 измерены; для 10000 приведена экстраполяция из-за длительного времени выполнения.

Таблица 1 - Результаты измерения

N	Общее время, с	Умножение, с	Статус
100	0,002	0,001	измерено
200	0,012	0,011	измерено
400	0,096	0,094	измерено
1000	1,701	1,684	измерено
2000	16,681	16,611	измерено
4000	186,677	186,431	измерено
10000	≈ 2917	≈ 2913	оценка

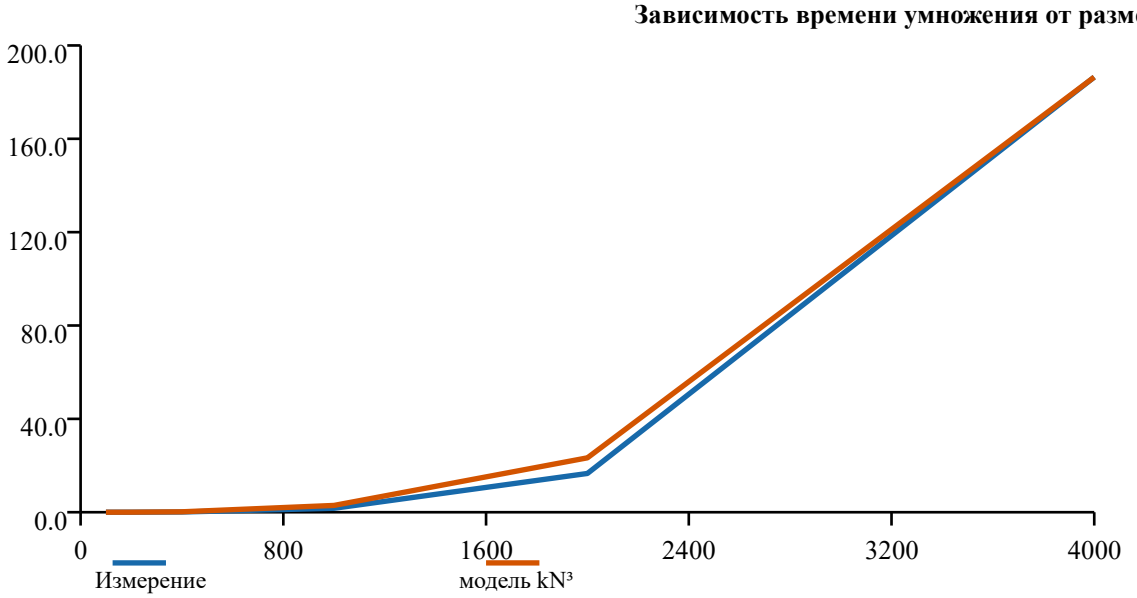
Оценка для N = 10000 получена по модели времени умножения, откалиброванной при N = 4000; прогноз составляет около 49 минут и не является измеренным результатом.

Сравнение с теоретической оценкой

Коэффициент модели определён по измерению умножения при N = 4000: $k = 186{,}431 / 4000^3 \approx 2{,}913 \times 10^{-9}$ с. Модель времени умножения: $T(N) = kN^3$.

Таблица 2 - Эксперимент и модель

N	Измеренное время, с	kN^3, с	Отклонение, с
100	0,001	0,003	-0,002
200	0,011	0,023	-0,012
400	0,094	0,186	-0,092
1000	1,684	2,913	-1,229
2000	16,611	23,304	-6,693
4000	186,431	186,431	+0,000



На измеренных размерах заметен кубический рост, ожидаемый для тройного вложенного цикла. Отклонения от модели связаны с особенностями кэша, накладными расходами и изменением условий измерения. Модель откалибрована по измерению при N = 4000; прогноз при N = 10000 следует считать ориентировочным.

Псевдокод

```
АЛГОРИТМ УмножениеМатриц(N)
выделить A, B, C размером N×N
заполнить A и B случайными числами
ДЛЯ i = 0..N−1; ДЛЯ j = 0..N−1: sum := 0
ДЛЯ r = 0..N−1: sum := sum + A[i,r]·B[r,j]
C[i,j] := sum освободить A, B, C
КОНЕЦ
```

Задание 2

Реализованы сортировка Шелла с последовательностью шагов {9, 5, 3, 2, 1}, собственная быстрая сортировка и стандартная функция qsort. Для честного сравнения перед каждым запуском рабочий массив копируется из одинакового исходного набора, а после сортировки проверяется неубывающий порядок.

Проверяются четыре входных набора: случайный, возрастающий, убывающий и массив с возрастающей первой и убывающей второй половиной. Замеры выполнены при N = 10000 в отладочной сборке; время каждого алгоритма усреднено по 10 запускам. Копирование исходного массива не входит в измеряемый интервал.

Таблица 3 - Среднее время сортировки, с (N = 10000)

Тип исходного массива	Shell	Quick	qsort
Случайный	0,0042	0,0005	0,0010
Возрастающий	0,0001	0,0001	0,0002

Убывающий	0,0080	0,0001	0,0003
Смешанный	0,0041	0,0187	0,0008

В этой сборке быстрая сортировка на смешанном массиве заметно медленнее: выбор среднего по индексу элемента даёт неравномерные разбиения для такого входа. Сортировка Шелла лучше проявляет себя на уже возрастающем массиве. Результаты зависят от процессора, компилятора и режима сборки; значения clock() округлены до точности часов.

Псевдокод сортировки Шелла

```
АЛГОРИТМ ShellSort(A, N)
K := [9, 5, 3, 2, 1]
ДЛЯ каждого gap из K:
ДЛЯ i = gap..N−1: x := A[i]; j := i
ПОКА j ≥ gap И x < A[j−gap]: A[j] := A[j−gap]; j := j−gap
A[j] := x
КОНЕЦ
```

Псевдокод быстрой сортировки и измерения

```
АЛГОРИТМ QuickSort(A, left, right)
i := left; j := right; pivot := A[(left + right) / 2] ПОКА i ≤ j:
ПОКА A[i] < pivot: i := i + 1
ПОКА pivot < A[j]: j := j − 1
ЕСЛИ i ≤ j: обменять A[i], A[j]; i := i + 1; j := j − 1
ЕСЛИ left < j: QuickSort(A, left, j)
ЕСЛИ i < right: QuickSort(A, i, right)
КОНЕЦ

АЛГОРИТМ Measure(сортировка, source, work, N)
скопировать source в work
t1 := clock(); сортировка(work, N); t2 := clock()
проверить, что work отсортирован
вернуть (t2 − t1) / CLOCKS_PER_SEC
КОНЕЦ
```

Вывод

Цель работы достигнута. Умножение матриц имеет сложность $O(N^3)$, а хранение трёх матриц требует $O(N^2)$ памяти. Время измерено для размеров до 4000; время при N = 10000 оценено по модели и не выдаётся за эксперимент. Сравнение сортировок показало зависимость времени от исходного порядка. На смешанном массиве используемая быстрая сортировка оказалась существенно медленнее, что связано с выбором опорного элемента.